

Quick Sort

O **Quick Sort** é um algoritmo de ordenação eficiente baseado na estratégia **Divisão e Conquista** (Divide and Conquer). Criado por Tony Hoare em 1959, é amplamente utilizado na prática devido ao seu excelente desempenho médio e uso eficiente de memória.

A ideia principal é escolher um elemento chamado **pivô** e reorganizar o vetor (particionamento) de forma que:

- Elementos menores ou iguais ao pivô fiquem à sua esquerda.
- Elementos maiores que o pivô fiquem à sua direita.
- O pivô fique em sua posição final e correta na ordenação.

Depois disso, o processo é repetido recursivamente para as duas partes do vetor (à esquerda e à direita do pivô).

Funcionamento

O algoritmo segue três etapas fundamentais:

1. **Escolha do pivô:** Seleciona-se um elemento do vetor (ex: primeiro elemento, último, elemento central ou pivô aleatório).
2. **Particionamento:** Reorganiza-se o vetor dividindo os elementos em relação ao pivô.
3. **Recursão:** Aplica-se o Quick Sort recursivamente na sublista da esquerda e na sublista da direita.

Caso	Complexidade de Tempo	Descrição
Melhor Caso	$\mathcal{O}(n \log n)$	O pivô sempre divide o vetor ao meio (divisão equilibrada).
Caso Médio	$\mathcal{O}(n \log n)$	Ocorre na grande maioria das distribuições de dados na prática.
Pior Caso	$\mathcal{O}(n^2)$	O pivô escolhido é sempre o maior ou menor elemento (ex: vetor já ordenado).

Exemplo Prático de Particionamento

Considere o vetor inicial: [3, 8, 2, 5, 1, 4] e a escolha do **último elemento (4)** como **pivô**.

1. **Início:** Vetor [3, 8, 2, 5, 1, 4], Pivô = 4.
2. **Comparação e Trocas:**
 - 3 < 4 → Mantém à esquerda.
 - 8 > 4 → Fica à direita.
 - 2 < 4 → Move para a esquerda.
 - 5 > 4 → Fica à direita.

- $1 < 4 \rightarrow$ Move para a esquerda.
- 3. **Posicionamento do Pivô:** O pivô 4 troca de lugar para ficar entre os menores e os maiores.
- 4. **Resultado da etapa:** [3, 2, 1, 4, 8, 5]
 - **Esquerda (menores):** [3, 2, 1]
 - **Pivô:** 4 (em posição definitiva)
 - **Direita (maiores):** [8, 5]

Agora, o algoritmo é chamado recursivamente para [3, 2, 1] e [8, 5].

```
void trocar(int &a, int &b)
{
    int temp = a;
    a = b;
    b = temp;
}
int particionar(int vetor[], int inicio, int fim)
{
    int pivo = vetor[fim];
    int i = inicio - 1;
    for (int j = inicio; j < fim; j++)
    {
        if (vetor[j] <= pivo)
        {
            i++;
            trocar(vetor[i], vetor[j]);
        }
    }
    trocar(vetor[i + 1], vetor[fim]);
    return i + 1;
}
void quickSort(int vetor[], int inicio, int fim)
{
    if (inicio < fim)
    {
        int posicaoPivo = particionar(vetor, inicio, fim);
        quickSort(vetor, inicio, posicaoPivo - 1);
        quickSort(vetor, posicaoPivo + 1, fim);
    }
}
```